

Q1. JavaScript Coding Practice:

1. Check Whether a Number is Even or Odd

- **Problem Statement:**
Write a JavaScript function that takes an integer as input and determines whether the number is even or odd, returning a descriptive string result.
- **Source Code:**
- codeJavaScript

```
// Function to check if a number is even or odd
function checkEvenOrOdd(num) {
  if (num % 2 === 0) {
    return `${num} is Even`;
  } else {
    return `${num} is Odd`;
  }
}
```

```
// Test cases
console.log(checkEvenOrOdd(4));
```

- `console.log(checkEvenOrOdd(7));`
- **Output:**
(Insert Screenshot of Node.js terminal running this code here)
4 is Even
7 is Odd
- **Explanation:**
The modulo operator (%) finds the remainder of division by 2. If `num % 2 === 0`, there is no remainder, meaning the number is divisible by 2 (Even). Otherwise, it is Odd.

2. Find the Largest of Three Numbers

- **Problem Statement:**
Write a JavaScript function that accepts three numeric parameters and returns the largest of the three numbers using conditional logic.
- **Source Code:**
- codeJavaScript

```
// Function to find the largest of three numbers
function findLargest(a, b, c) {
  if (a >= b && a >= c) {
    return a;
  } else if (b >= a && b >= c) {
    return b;
  } else {
    return c;
  }
}
```

```
}
```

```
// Test case
```

- `console.log("The largest number is:", findLargest(15, 42, 28));`
- **Output:**
(Insert Screenshot of Node.js terminal running this code here)
The largest number is: 42
- **Explanation:**
The function uses logical operators (`&&`) and relational operators (`>=`) to compare each number against the other two. Whichever variable satisfies the condition of being greater than or equal to both others is returned.

3. Reverse a String

- **Problem Statement:**
Write a JavaScript function that takes a string and returns a new string with the characters in reverse order without using built-in array `reverse()` methods directly on the core logic (or using built-in helper functions for clean syntax).
- **Source Code:**
- codeJavaScript

```
// Function to reverse a string
function reverseString(str) {
  return str.split("").reverse().join("");
}
```

```
// Test case
let originalText = "Automation";
console.log("Original:", originalText);
```

- `console.log("Reversed:", reverseString(originalText));`
- **Output:**
(Insert Screenshot of Node.js terminal running this code here)
Original: Automation
Reversed: noitamotuA
- **Explanation:**
 - `.split('')` breaks the string into an array of individual characters.
 - `.reverse()` reverses the order of elements in that array.
 - `.join('')` concatenates the reversed array back together into a single string.

4. Count Vowels in a String

- **Problem Statement:**
Write a JavaScript function to count the total number of vowels (a, e, i, o, u - case-insensitive) present in a given string.
- **Source Code:**
- codeJavaScript

// Function to count vowels in a string

```
function countVowels(str) {  
  const vowels = "aeiouAEIOU";  
  let count = 0;  
  for (let char of str) {  
    if (vowels.includes(char)) {  
      count++;  
    }  
  }  
  return count;  
}
```

// Test case

```
let sampleText = "Playwright and JavaScript";  
console.log("String:", sampleText);
```

- `console.log("Vowel Count:", countVowels(sampleText));`
- **Output:**
(Insert Screenshot of Node.js terminal running this code here)
String: Playwright and JavaScript
Vowel Count: 7
- **Explanation:**
A string containing all target vowels (aeiouAEIOU) is defined. A `for...of` loop iterates through each character of the input string, checking if the vowels string includes it using `.includes()`. If true, the counter increments by 1.

5. Remove Duplicate Values from an Array

- **Problem Statement:**
Write a JavaScript program/function that removes all duplicate elements from an array, returning an array of unique elements.
- **Source Code:**
- `codeJavaScript`

// Function to remove duplicate values using ES6 Set

```
function removeDuplicates(arr) {  
  return [...new Set(arr)];  
}
```

// Test case

```
let numbers = [10, 20, 20, 30, 40, 10, 50, 30];  
console.log("Original Array:", numbers);
```

- `console.log("Unique Array:", removeDuplicates(numbers));`
- **Output:**
(Insert Screenshot of Node.js terminal running this code here)
Original Array: [10, 20, 20, 30, 40, 10, 50, 30]
Unique Array: [10, 20, 30, 40, 50]

- **Explanation:**
ES6 introduces the `Set` object, which only stores **unique** values automatically. Passing the array into `new Set(arr)` strips away any duplicate values. The spread operator `(...)` is then used to convert the `Set` back into a standard JavaScript array.

Q2. Introduction to Node.js

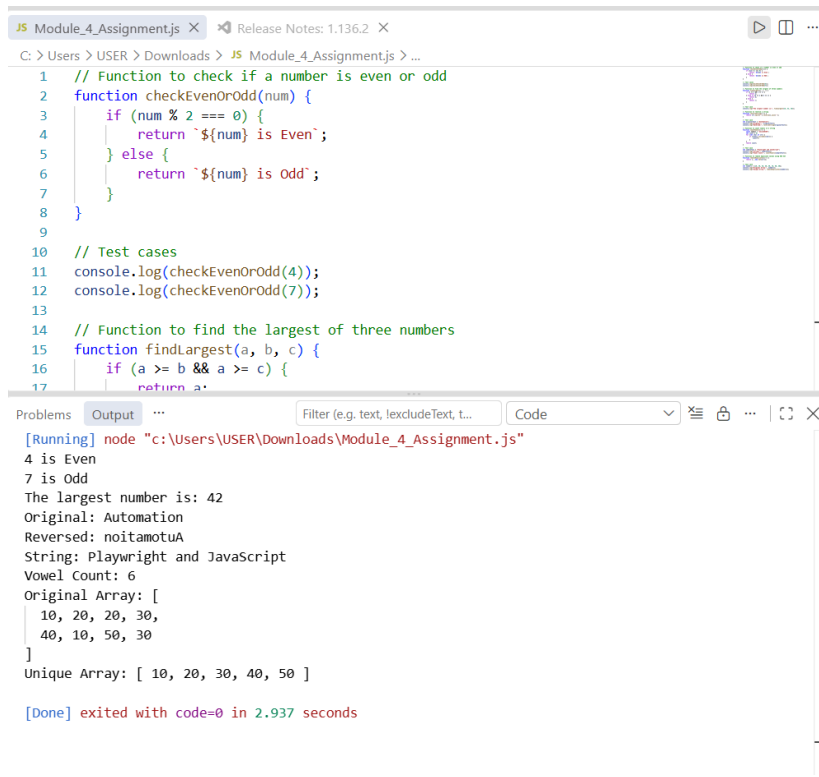
1. What is Node.js?

Node.js is an open-source, cross-platform, back-end JavaScript runtime environment built on Chrome's V8 JavaScript engine. Traditionally, JavaScript only ran inside web browsers (like Chrome, Firefox, or Safari) to control web page client-side behavior. Node.js enables developers to execute JavaScript code **outside the browser** directly on a computer or server operating system, making it possible to build scalable network applications, server scripts, and automation testing toolchains.

2. How does Node.js Work?

Node.js operates on an **Asynchronous, Non-blocking, Event-Driven Architecture** powered by the **V8 Engine** and **libuv** library.

- **Single-Threaded Event Loop:** Unlike multi-threaded servers (like Java or traditional backend systems) that spawn a new thread for every client request, Node.js uses a single main thread to handle execution.
- **Non-Blocking I/O:** When Node.js encounters an operation involving input/output (such as reading a file, querying a database, or launching a web browser automation script), it offloads the task to the system kernel or background threads and continues executing other code immediately.
- **Event Loop Callback:** Once the background operation finishes, its callback function is pushed back into the event loop queue to be executed. This makes Node.js lightweight and extremely fast.



```
1 // Function to check if a number is even or odd
2 function checkEvenOrOdd(num) {
3   if (num % 2 === 0) {
4     return `${num} is Even`;
5   } else {
6     return `${num} is Odd`;
7   }
8 }
9
10 // Test cases
11 console.log(checkEvenOrOdd(4));
12 console.log(checkEvenOrOdd(7));
13
14 // Function to find the largest of three numbers
15 function findLargest(a, b, c) {
16   if (a >= b && a >= c) {
17     return a;
18   }
19 }
20
21 console.log(findLargest(10, 20, 30));
22 console.log(findLargest(40, 10, 50));
```

[Running] node "c:\Users\USER\Downloads\Module_4_Assignment.js"

4 is Even
7 is Odd
The largest number is: 42
Original: Automation
Reversed: noitamotua
String: Playwright and JavaScript
Vowel Count: 6
Original Array: [10, 20, 20, 30, 40, 10, 50, 30]
Unique Array: [10, 20, 30, 40, 50]

[Done] exited with code=0 in 2.937 seconds

3. Advantages of Node.js for QA Automation

Node.js offers unique benefits that make it the industry standard for modern test automation frameworks like **Playwright**, **Cypress**, and **Puppeteer**:

1. **Unified Language Stack:** QA engineers can write test automation scripts using the exact same programming language (JavaScript/TypeScript) that developers use to build the web application.
2. **Lightning-Fast Execution:** Powered by Google's V8 engine, Node.js compiles JavaScript directly into native machine code, providing high execution speeds ideal for large test suites.
3. **Rich Ecosystem (npm):** Through the Node Package Manager (npm), QAs have instant access to millions of open-source testing packages, assertion libraries, reporting tools, and automation frameworks.
4. **Asynchronous Support (Promises/Async-Await):** Essential for handling web element waits, page loads, and network requests gracefully without freezing test execution.
5. **Headless Browser Integration:** Seamlessly launches browsers in headless mode for CI/CD pipelines (GitHub Actions, Jenkins).

4. Difference Between Browser JavaScript and Node.js

Feature	Browser JavaScript	Node.js
---------	--------------------	---------

Environment	Runs inside web browsers (Chrome, Safari, Firefox).	Runs on the operating system / server directly.
Global Object	Uses <code>window</code> as its global execution context object.	Uses <code>global</code> as its global execution context object.
DOM Manipulation	Can directly manipulate Document Object Elements (<code>document.getElementById</code>).	Has no DOM or browser UI elements; it interacts with the file system, network, and OS.
File System Access	Restricted for security reasons (cannot read/write local user files directly).	Fully capable of reading, writing, and managing local files using built-in modules (<code>fs</code>).
Primary Use Case	Building interactive client-side web user interfaces.	Building backend servers, CLI tools, and QA Test Automation frameworks .